AUTOMATED REVIEW RATING SYSTEM

BALANCED DATASET

Jeevan Santhosh S

PROJECT OVERVIEW

An intelligent system that analyzes customer reviews and predicts corresponding star ratings. It uses natural language processing (NLP) to extract sentiment and key features from textual feedback. The model is trained on labelled review data to learn patterns between language and rating.

ENVIRONMENT SETUP

- Installed Python 3.10+ with required libraries: pandas, numpy, matplotlib, seaborn, spacy, scikit-learn, beautifulsoup4, requests.

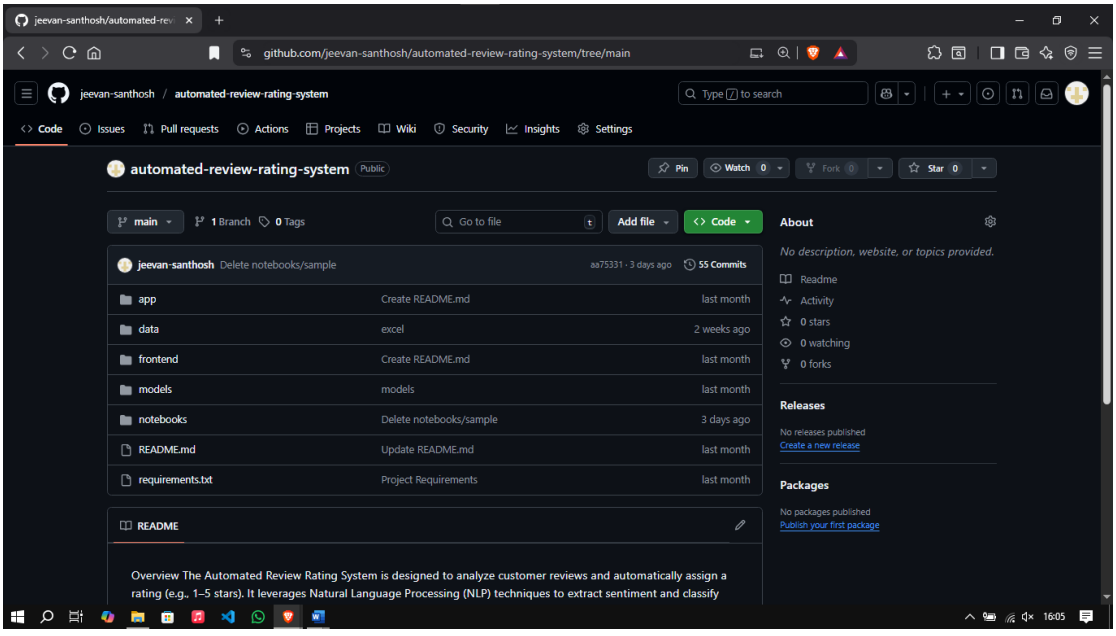
- IDE used: VS Code

- Verified proper functioning of packages for data preprocessing, NLP, and visualization.

GITHUB PROJECT SETUP

The project was set up on GitHub, a new repository was created to store datasets, preprocessing scripts, and model files. The setup process included initializing the repository, configuring Git on the local system, and pushing the project files to GitHub. A screenshot of the GitHub repository setup is included below to demonstrate the successful configuration and repository structure for the Automated Review Rating System. Link of GitHub repository **automated-review-rating-system.**

Structure of the directory :



DATA COLLECTION

- 3 Datasets were collected through Kaggle.
- Datasets collected from Kaggle when combined had 2,01,538 rows and these were datasets related with flipkart reviews.
- Dataset link – Expanded Dataset

DATA PREPROCESSING

Effective data preprocessing is essential to improve the performance and accuracy of machine learning models. The following techniques were applied to prepare the raw review dataset for modeling.

1. Removing Duplicates

Duplicate rows containing the exact same review and rating were removed to prevent bias and overfitting. This ensured that the dataset only included unique observations.

Code :

```
expanded_dataset.drop_duplicates(subset=["Review"], inplace=True)
```

2. Removing Conflicting Reviews

Some reviews had identical text but different star ratings. These inconsistencies can confuse the model. Such conflicting entries were identified and removed to maintain label clarity.

Code :

```
conflict_reviews =
expanded_dataset.groupby("Review")["Rating"].nunique()
conflict_reviews = conflict_reviews[conflict_reviews > 1].index

expanded_dataset =
expanded_dataset[~expanded_dataset["Review"].isin(conflict_reviews)]
```

3. Handling Missing Values

Rows with missing or null values-particularly in the review text or rating column-were removed. This step ensured the dataset was complete and meaningful for analysis.

Code :

```
expanded_dataset.dropna(subset=["Review", "Rating"], inplace=True)
```

4. Dropping Unnecessary Columns

Non-essential columns such as review IDs, timestamps, or user identifiers were dropped. These fields did not contribute to the model and could introduce noise or privacy concerns. Rows with missing or null values particularly in the review text or rating column were removed. This step ensured the dataset was complete and meaningful for analysis

Code:

```
expanded_dataset = expanded_dataset[["ID", "Rating", "Review"]].copy()
```

5. Lowercasing Text

All review text was converted to lowercase to maintain uniformity. This helps prevent duplication of tokens like "Good" and "good" being treated as separate words.

Code :

```
text = str(text).lower()
```

6. Removing URLs

URLs present in the review text were removed using regular expressions.

Code :

```
text = re.sub(r'http\S+|www.\S+', '', text)
```

7. Removing Emojis and Special Characters

Emojis and special symbols may not contribute meaningful context for text analysis (unless specifically required). We remove these characters to focus on the core textual content.

Code :

```
emoji_pattern = re.compile("["
                                u"\U0001F600-\U0001F64F"
                                u"\U0001F300-\U0001F5FF"
                                u"\U0001F680-\U0001F6FF"
                                u"\U0001F1E0-\U0001F1FF"
```

```
                                "]+", flags=re.UNICODE)
    text = emoji_pattern.sub(r'', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

8. Removing Puncuations

Punctuation marks and numbers can disrupt the natural language patterns of the text. By removing then, we simplify the tokenization process and prevent punctuation from being misinterpreted as separate tokens.

Code :

```
text = re.sub(r'<.*?>', '', text)
```

9. Stopwords Removal

Stopwords, such as "the", "is", and "and", are frequently occurring words that might not add significant semantic value. We use libraries like Spacy to filter these words out, reducing noise and focusing on words that contribute meaning to the reviews, there were total 175 stop words in spacy.

```
("under", "it", "for", "is", "but", "to", "its", "very", "this", "with",
"so", "if", "any", "has", "that", "s", "of", "can", "only", "i", "you",
"are", "or", "my", "am", "over", "all", "a", "not", "in", "was", "and",
"few", "other", "same", "about", "these", "doesn", "t", "m", "ll",
"again", "after", "as", "now", "the", "because", "doing", "more", "than",
"just", "don", "have", "at", "when", "up", "some", "will", "who", "no",
"on", "didn", "why", "from", "re", "they", "here", "be", "while",
"there", "which", "we", "should", "what", "between", "too", "once", "me",
"your", "then", "our", "been", "how", "an", "off", "d", "o", "during",
"nor", "their", "y", "won", "ve", "above", "both", "having", "below",
"such", "most", "down", "being", "until", "each", "couldn", "out",
"into", "before", "through", "further", "yours", "her", "she", "he",
"had", "them", "himself", "where", "those", "them", "own", "myself",
"itself", "yourselves", "themselves", "haven", "isn", "aren", "wouldn",
"shouldn", "hadn", "does", "did", "doing", "done", "having")
```

Code:

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import spacy

nlp = spacy.load('en_core_web_sm')
nltk_stopwords = set(stopwords.words('english'))
```

```
def preprocess_text(text):
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if token.text not in
nltk_stopwords and token.is_alpha]
    return " ".join(tokens)
```

10. Lemmatization

Lemmatization is a text preprocessing technique that reduces words to their base or dictionary form, known as the lemma. For example, words like "running", "ran", and "runs" are all reduced to the base form "run". Unlike stemming, lemmatization uses linguistic rules and vocabulary, ensuring that the resulting word is meaningful. This helps in grouping similar words and improves model performance by reducing the size of the vocabulary without losing context.

Why lemmatization is better than stemming?

Lemmatization is often preferred over stemming because:

1. Produces Real Words:
Lemmatization returns valid dictionary words (e.g., "running" "run"), whereas stemming may return incomplete or non-existent words (e.g., "running" → "runn").

2. Context-Aware:
Lemmatization considers the context and part of speech (POS) of a word to find its correct root form.
Stemming blindly trims word endings without understanding grammar.

3. More Accurate and Clean:
Lemmatization provides cleaner and more accurate tokens, which improves model understanding and performance, especially in NLP tasks like sentiment analysis or classification.

Code :

```
def preprocess_text(text):
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if token.text not in
nltk_stopwords and token.is_alpha]
    return " ".join(tokens)
```

11. Filtering by Word Count

Very short reviews might not contain enough context to be useful, and excessively long reviews could be outliers. We apply filtering to exclude:

- Reviews with fewer than 3 words.
- Reviews that exceed 100 words.
  This ensures that the dataset remains robust and relevant for model training.

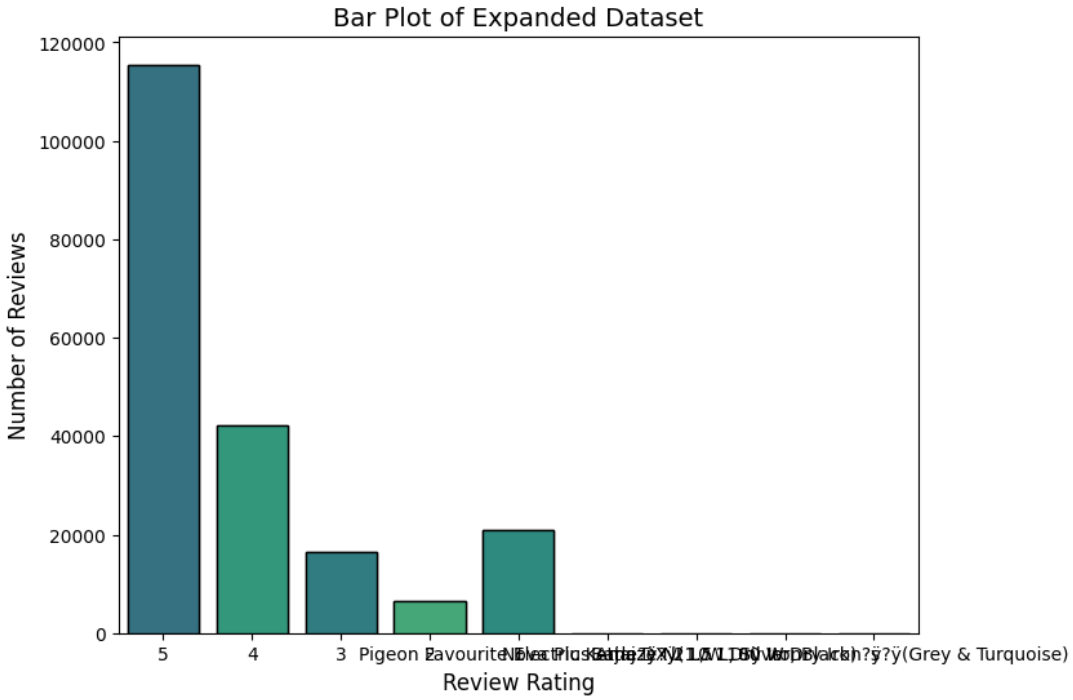
Code :

```
before_word_filter = len(expanded_dataset)

  expanded_dataset = expanded_dataset[expanded_dataset["Review"].apply
                              (lambda x: 3 <= len(str(x).split()) <=
                              10)]

    after_word_filter = len(expanded_dataset)
```

DATA VISUALIZATION

Bar Plot

A bar plot is a chart that represents categorical data with rectangular bars, where the height of each bar indicates the value or frequency of that category. In this project, bar plots were used to show the number of reviews per rating class (1 to 5 stars), helping to visualize class distribution and detect any imbalance in the dataset.



BALANCED DATASET

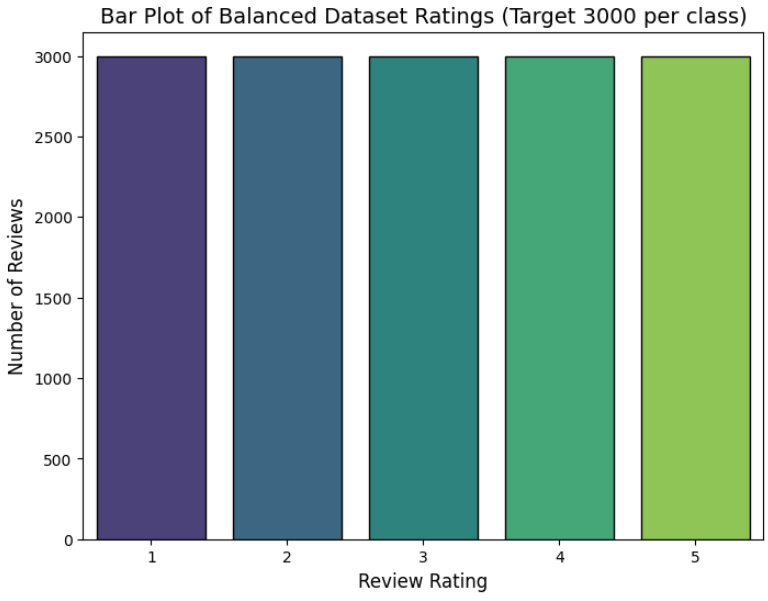
To ensure that the model is trained on an unbiased dataset, a balanced dataset was created by selecting **3000 reviews for each rating** from 1 to 5. This resulted in a **total of 15,000 reviews**, ensuring equal representation across all rating categories.

Code :

```
target_count = 3000
balanced_data = []

ratings = df_filtered['Rating'].unique()

for rating in sorted(ratings):
    rating_subset = df_filtered[df_filtered['Rating'] == rating]

    sample_size = min(len(rating_subset), target_count)

    sampled_subset = rating_subset.sample(n=sample_size,
random_state=42)
    balanced_data.append(sampled_subset)

df_balanced = pd.concat(balanced_data).reset_index(drop=True)

print(f"Balanced dataset created. Total reviews:
{df_balanced.shape[0]}")
print("Final count per rating in balanced dataset:")
print(df_balanced['Rating'].value_counts().sort_index())
```

1. Data Visualization of Balanced Dataset

Bar Plot of Balanced Dataset :



2. Sample Reviews of Balanced Dataset

- 1 STAR (5 reviews)
    1. worst model user please buy
    2. inferior quality looks cheap
    3. bad product product stopped working switch hold soon one presses
    4. wood part broken
    5. worst materials waste money

- 2 STAR (5 reviews)
    1. sole shoes badand fabric quality cheap recommend buy product going return
    2. qulity use befoure packad item verry bead qulity
    3. print proper fabric also per expectations
    4. bad product bad sound quality
    5. bat quality buy product flipkart return money

- 3 STAR (5 reviews)
    1. quiet uncertain inverter okay power button work connecting battery soon wire connected battery inverter starts switch dont know z z z z z z
    2. satisfied battery backup else everything good
    3. awesome worth money good bass nice sound clarity
    4. good product value money thanks filipkart team
    5. awesome product thank flipkart trust v gaurd super product

- 4 STAR (5 reviews)
    1. thanks flipkart team wonderful delivery time kind delivery boy product purchased mom happy gifted mom birthday happy amazing product realme awesome built qualityb best battery little bit heavy fine handle

afterall mh battery c camera good front back better maybe updates display also good thanks

2. boat bassheads decent sound quality bass boat known low give u perfect sound quality still good price segment

3. writing review week use first run machine need additional suppliments like dishwasher salt rinse aid dishwasher detergent however small kit comes machine one use come washing washing utensils effective steel utensils shine like never washing time eco mode hrs quick mode mins always use eco mode uses.

4. worth money spent nice

5. guys using dsl almost months image quality good good lighting conditions much better mobile phone cameras indoor performance also good flash use without flash low lighting condition pros decent image quality price range decent battery life take photos single charger wi fi function canon app download

- 5 STAR (5 reviews)

1. cheapest mechanical keyboard market go using programming work keys feel good

2. wow great itemperfect packthanks seller flipkart spcila thnx ekart guy badhusha trivandrum

3. amazing wall clock lowest price

4. dry yeast really pure gets activated soon warm water really helpful baking makes bread soft smooth useful

5. really liked bought father law size good

TRAIN TEST SPLIT

Train-Test Split is a technique to divide the dataset into two separate sets:

- Training Set (typically 80%): Used to train the machine learning model.
- Test Set (typically 20%): Used to evaluate the model's performance on unseen data.

This separation ensures that the model generalizes well and is not just memorizing the training data.

Why Stratified Split?

In classification problems like review rating prediction, stratified splitting is important to maintain class balance (equal distribution of ratings) in both training and testing sets.

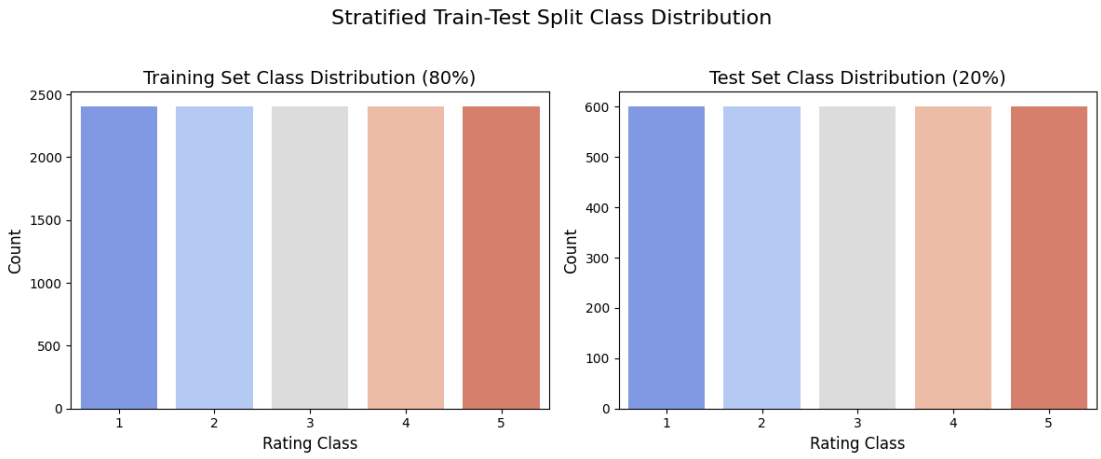
Without stratification, some rating classes (like 1-star or 5-star) may be underrepresented in the test set, leading to biased evaluation.

Code :

```
# Stratified split (80% train, 20% test)
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y,
    test_size=0.2,
    stratify=y, # Ensures the class proportion is maintained in both splits
    random_state=42
```

How It Was Done:

To prepare the data for model training, the dataset was first shuffled randomly to eliminate any order bias. A stratified train-test split was then performed using train teat split from rkimarn.model selection with the ratify-y argument to ensure that all star ratings were proportionally represented in both training and testing sets. The dataset was split using an 80% training and 20% testing ratio. After splitting, all text preprocessing stops-Including lowercasing. Iemmatization, stopword removal, and cleaning were applied separately tox train and x text to prevent data leakage and maintain model integrity.



TEXT VECTORIZATION

Machine learning models can't work directly with raw text-they require numerical input. Vectorization is the process of converting text data into numerical features.

TF-IDF (Term Frequency - Inverse Document Frequency)

TF-IDF is a statistical technique that represents how important a word is to a document relative to the entire corpus.

- TF (Term Frequency): How often a word appears in a document.
- IDF (Inverse Document Frequency): Penalizes common words and highlights rare but Important ones.

This approach helps to capture both word relevance and discriminative power, making it better than simple word counts.

Formula for TF-IDF:

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF-IDF(t,d) = TF(t,d) * IDF(t)$$

Code :

```
tfidf = TfidfVectorizer(stop_words='english', max_features=10000,
ngram_range=(1, 2))
X_train = tfidf.fit_transform(X_train_text)
X_test = tfidf.transform(X_test_text)
```

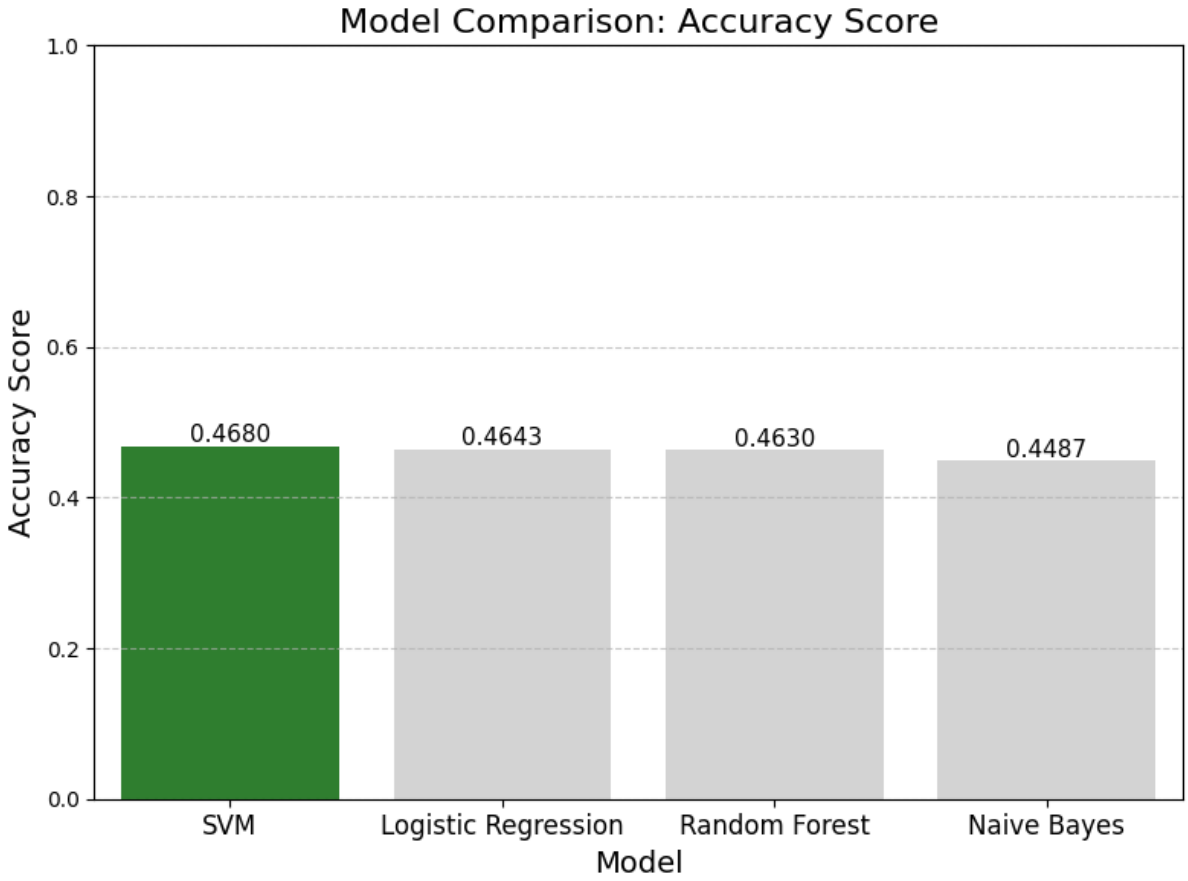
MODEL PERFORMANCE ON TRAINING

The evaluation of multiple machine learning models trained on a balanced text review dataset to predict sentiment or rating categories. The balanced dataset ensures equal representation of all classes, allowing a fair comparison of model performance without bias toward dominant classes. Four models are Logistic Regression, Naive Bayes, Random

Forest, and SVM were assessed using key metrics such as accuracy, precision, recall, and F1-score to determine their effectiveness and generalization capability for text classification tasks.

MODEL	ACCURACY	PRECISION	RECALL	F1-SCORE
Logistic Regression	0.4643	0.58	0.64	0.61
Naive Bayes	0.4487	0.57	063	0.60
Random Forest	0.4630	0.58	0.63	0.60
SVM	0.4680	0.60	0.59	0.60

Among all models trained on the balanced dataset, **SVM** achieved the highest overall performance with an accuracy of **0.4680**, followed closely by Logistic Regression at 0.4643. Both models demonstrated strong precision-recall balance, indicating effective generalization across all rating classes. Tree-based models like Random Forest showed lower performance, suggesting that linear models are better suited for the TF-IDF-based text representation used in this dataset.



CROSS-TEST (Balanced - Imbalanced)

When the model trained on the balanced dataset was tested on the imbalanced dataset, it achieved a **cross-test accuracy of 0.7913**, with a **weighted precision of 0.7854** and a **weighted F1 score of 0.8308**. These results indicate that the model generalizes reasonably well to a different data distribution, maintaining strong overall performance despite class imbalance.

Which model would you deploy and why?

Based on the evaluation and fine-tuning results, SVM is the recommended model for deployment. It achieved the highest accuracy (0.4680) on the balanced dataset, maintained strong performance during cross-testing on the imbalanced dataset (accuracy 0.7913), and demonstrated a good balance of precision and F1-score. SVM is well-suited for high-dimensional TF-IDF text features, providing robust generalization across classes. Its linear decision boundary ensures efficient computation and scalability, making it the best choice for reliable, production-ready text classification in this scenario.

UI & DEPLOYMENT

For UI creation I used Streamlit. It provides an interactive interface to analyze customer reviews using deep learning models. Users can input any review text, and the app instantly displays predicted ratings from two trained models one trained on balanced data and another on imbalanced data. It visually compares their outputs, confidence scores, and overall reliability. This prototype helps demonstrate how dataset balance affects model performance and prediction stability, offering a user-friendly way to test and evaluate review sentiment classification in real time.